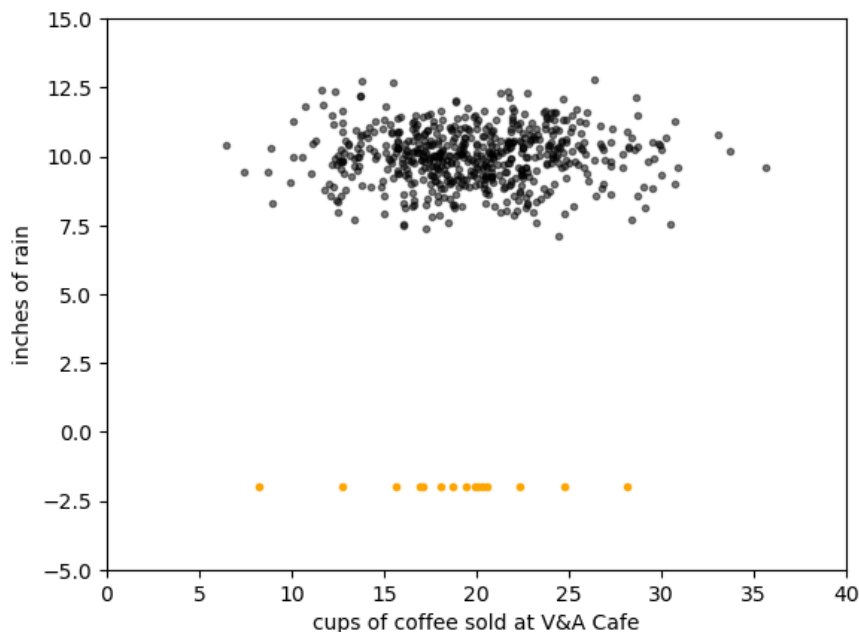


Discussion #3 Solutions

Note: Your TA will probably not cover all the problems. This is totally fine; the discussion worksheets are not designed to be finished within an hour. They are deliberately made slightly longer so they can serve as resources you can use to practice, reinforce, and build upon concepts discussed in lectures, labs, and homework.

Dealing with Missing Data

1. Suppose that you plot “cups of coffee sold at V&A Cafe per day” versus “inches of rain per day” across a period of two months, shown below. V&A Cafe is not missing any data, but 30% of the data in “inches of rain” are NaN values that have been represented with “-2”, an impossible amount of rain.



Which of the following techniques would be **most** effective in solving the issue of missing data?

- ☐ A. Using the mean to impute the missing values
- ☐ B. Using the mode(s) to impute the missing values
- ☐ C. Using the median to impute the missing values
- ☐ D. Dropping any rows with missing values

- ☐ E. Imputing missing values with zero

Solution:

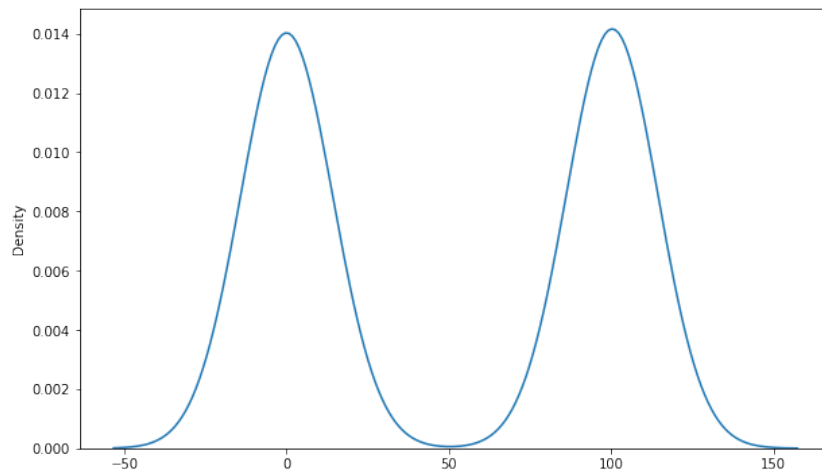
A, C: Since the distribution of the y-axis is roughly symmetric around the center, replacing the NaN values with 10 is reasonable. Thus, the mean and median would work.

B: The mode isn't meaningful due to lack of "exact" repeated values.

D: Remember that 30% of the data is missing, and it's possible that the data could be missing due to an inherent bias. Hence, dropping the NaN values could significantly affect the data.

E: It is unreasonable to input with zero here, additionally given there are no zero observations in the plot.

2. Suppose we examine the amount of money lost and gained in a game of poker, and see that this variable is missing 1% of its values. Its distribution, shown below, is constructed from all valid (non-NaN) values, in cents.



Which of the following techniques would be **reasonably** effective in solving this issue of missing data?

- ☐ A. Using the mean to impute the missing values
- ☐ B. Using the mode(s) to impute the missing values
- ☐ C. Using the median to impute the missing values
- ☐ D. Dropping any rows with missing values
- ☐ E. Imputing missing values with zero

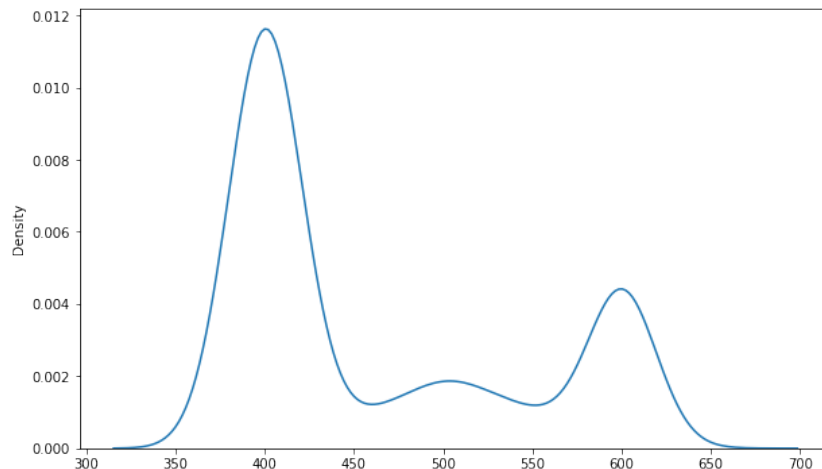
Solution:

A, C: This is a bimodal distribution, so the mean or median would yield values around 50, where the least amount of data is concentrated. That would not be a good representation of this distribution.

B, E: The shape shows that there are 2 modes at 0 and 100. Both represent where the majority of the data is concentrated, so using them to impute the missing values gives the best estimate.

D: Since only 1% of the data is missing, dropping the NaN values would not have an immense effect on the data. Additionally, there could be an underlying reason behind the two different modes, so it might be safer to drop NaN values than to (incorrectly) impute the wrong mode.

3. Suppose that while exploring a climate dataset with a million records, you realize that 20% of atmospheric measurements in different fields are NaN values! Its distribution, shown below, is constructed from all valid (non-NaN) values, in millibars.



Which of the following techniques would be **most** effective in solving this issue of missing data?

- ☐ A. Using the mean to impute the missing values
- ☒ B. Using the mode(s) to impute the missing values
- ☒ C. Using the median to impute the missing values
- ☐ D. Dropping any rows with missing values
- ☐ E. Imputing missing values with zero

Solution: A: The mean would likely yield a number in the middle rather than at the two “bigger” modes of the data. Hence, it wouldn’t be as effective.

B, C: The mode is effective again since that’s where the data is most concentrated, and the median would do a reasonable job since it ignores the tail of the distribution far better than the mean. We do not want to drop any rows by the same reasoning as for all sub-parts, and imputing values with zero adds another mode.

D, E: Dropping 20% of the data is a substantial proportion of the data that could introduce additional bias in analysis. Similarly, imputing with zeros given the range of values between 3000 and 700 is not a valid option.

Note: Not selecting the median as appropriate would be a reasonable conclusion, too, simply based on the figure. Mathematically, the median is around 409 for this figure, which is essentially the biggest mode, whereas the mean is 462, which is far worse.

Regular Expressions (Regex)

Regular Expressions (Regex for short) are an immensely powerful tool for parsing strings. However, its many rules make Regex very confusing, even for veteran users, so please don't hesitate to ask questions! Here's a snippet of the Regex portion of the Fall 2025 Final Reference Sheet:

Metacharacters

Regular Expressions

Operator	Description	Operator	Description
.	Matches any character except <code>\n</code>	*	Matches preceding character/group zero or more times
\	Escapes metacharacters	?	Matches preceding character/group zero or one times
	Matches expression on either side of expression; has lowest priority of any operator	+	Matches preceding character/group one or more times
\d, \w, \s	Predefined character group of digits (0-9), alphanumerics (a-z, A-Z, 0-9, and underscore), or whitespace, respectively	^, \$	Matches the beginning and end of the line, respectively
\D, \W, \S	Inverse sets of <code>\d</code> , <code>\w</code> , <code>\s</code> , respectively	()	Capturing group used to create a sub-expression
{m}	Matches preceding character/group exactly <code>m</code> times	[]	Character class used to match any of the specified characters or range (e.g. <code>[abcde]</code> is equivalent to <code>[a-e]</code>)
{m, n}	Matches preceding character/group at least <code>m</code> times and at most <code>n</code> times. If either <code>m</code> or <code>n</code> are omitted, set lower/upper bounds to 0 and ∞ , respectively	[^]	Invert character class; e.g. <code>[^a-c]</code> matches all characters except <code>a</code> , <code>b</code> , <code>c</code>

Useful re Functions

Function	Description
<code>re.match(pattern, string)</code>	Returns a match if zero or more characters at beginning of <code>string</code> matches <code>pattern</code> , else None
<code>re.search(pattern, string)</code>	Returns a match if zero or more characters anywhere in <code>string</code> matches <code>pattern</code> , else None
<code>re.findall(pattern, string)</code>	Returns a list of all non-overlapping matches of <code>pattern</code> in <code>string</code> (if none, returns empty list)
<code>re.sub(pattern, repl, string)</code>	Returns <code>string</code> after replacing all occurrences of <code>pattern</code> with <code>repl</code>

4. Which string contains a match for the following regular expression, "1+1\$"? The character "_" represents a single space.

☐ A. What_is_1+1 ☐ B. Make_a_wish_at_11:11 ☐ C. 111_Ways_to_Succeed

Solution: Recall that `1+` matches on *at least one* occurrence of the character `1`, and `$` marks the end of the string. So the ending "11" is matched.

5. Write a regular expression that matches a string which contains only one word containing only lowercase letters and numbers. This expression should also match with the empty

string.

Solution: `^[a-z0-9]*$`

6. Given `sometext = "I've got 10 eggs, 20 geese, and 30 giants."`, use `re.findall` to extract all the items and quantities from the string. The result should look like `['10 eggs', '20 geese', '30 giants']`. You may assume that a space separates quantity and type, and that each item ends in `s`.

Solution: `re.findall(r"\d+\s\w+", sometext)`

7. For each pattern specify the starting and ending position of the first match in the string. The index starts at zero and we are using closed intervals (both endpoints are included).

	abcdefg	abcs!	ab_abc	abc,_123
<code>abc*</code>	<code>[0, 2]</code>	<code>[0, 2]</code>	<code>[0, 1]</code>	<code>[0, 2]</code>
<code>[^\s]+</code>	<code>[0, 6]</code>	<code>[0, 4]</code>	<code>[0, 1]</code>	<code>[0, 3]</code>
<code>ab.*c</code>	<code>[0, 2]</code>	<code>[0, 2]</code>	<code>[0, 5]</code>	<code>[0, 2]</code>
<code>[a-z1,9]+</code>	<code>[0, 6]</code>	<code>[0, 3]</code>	<code>[0, 1]</code>	<code>[0, 3]</code>